



PLSPS 2026 Assignment

Parallelising a Lattice Boltzmann Fluid Simulation

Mikhail Cassar and Francesc Verdugo

2026

Contents

1 Overview	2
2 The simulation	3
3 Parallelisation approach	5
4 Tasks	5
5 Submission	7
6 Contact	8

1 Overview

This document describes the PLSPS 2026 assignment. It outlines the problem, the required parallelisation approach, the tasks, the grading scheme, and the deliverables. See the `README.md` file for setup instructions and the commands to run, test, and benchmark the code.

You work on this assignment in pairs. Form your group following in the instructions of the Canvas page for the assignment. All submissions are per group.

What you build

The project provides a complete, working sequential fluid simulation written in Julia. You must write a parallel solver, `run_par`, that uses the Message Passing Interface (MPI) to distribute the computation across several processes and produce the same result. After building the solver, you must measure its parallel performance and report your findings.

Note. Your group only writes one file: `src/run_par.jl`. Do not modify the `GeoFluidLBM.jl` file or the sequential solver (`run_seq`). You do not need to understand fluid dynamics to complete this assignment.

Grading

Each objective corresponds to one of the four tasks in Section 4. The following table lists the weight of each objective and the evidence we mark it against:

Objective	Evidence	Weight
Implementation	<code>run_par.jl</code> , Report section 1	35%
Verification	PASS at 2 and 4 ranks on DAS-5, Report section 2	Required
Benchmarking	Report section 3	35%
Analysis	Report section 4 and 5	30%

Implementation: We grade how well you describe the following aspects of the implementation: domain decomposition and work mapping, ghost column exchange, boundary conditions, and gathering the final result.

Verification: Your implementation must pass the provided tests at 2 and 4 ranks. After the deadline we re-run these tests on DAS-5 with your submitted `run_par.jl`; our run is the one that counts. If either fails, you score 0% for Implementation. Other objectives are unaffected.

Benchmarking: We grade the parallel performance of your code and how well you describe the experimental setup and the quality of the figures showing your results. To assess the parallel performance, we will compare the parallel speedup of your code against the teacher's implementation. The teacher implementation obtains parallel speedup close to the linear speedup for several (but not for all) of the test cases described below. If you never approach the linear speedup, there is something wrong with the parallel performance of your code.

Analysis: You should provide explanations for the effect of the grid size. For the cases that the speedup is significantly lower than the linear speedup, provide explanations about why this happens. In both cases, your explanations should combine hard evidence (e.g., computing communication over computation ratios) and be supported by the theory studied in the course.

Note. Use the L^AT_EX template provided in the folder `report` to write your report. Follow the structure given in the template.

Assignment grade: The objective weights give a score out of 100%. Your assignment grade is $A = 1 + 9 \times (\text{score}/100)$, so a submitted assignment is never graded below 1.

Reference material

This assignment adapts a concise Python implementation of the lattice Boltzmann method by Mora, Morra, and Yuen (2020)¹. We recommend reading Section 5 of their paper. It clearly explains the decomposition, the neighbour communication, and the gathering of results. **Reading the paper might be useful, but it is completely optional.**

Because the paper’s code is written in Python, its structure differs from the provided Julia solver. Use the paper as a guide for the parallelisation approach rather than a template to copy.

2 The simulation

The lattice Boltzmann method (LBM) is a computational fluid dynamics technique. LBM simulates fluid flow by tracking particle distribution functions on a fixed grid, rather than by solving the Navier–Stokes equations directly.

The scenario

The provided solver simulates pressure-driven channel flow, also known as Poiseuille flow. The domain is a two-dimensional channel with solid walls on the top and bottom rows.

The solver drives the flow by holding the inlet (left) column at a higher density than the outlet (right) column. The resulting pressure difference pushes fluid along the channel. At steady state, the fluid moves fastest in the centre and stops at the walls. The velocity profile across the channel forms an analytical parabola.

Figure 1 shows the flow field at steady state:

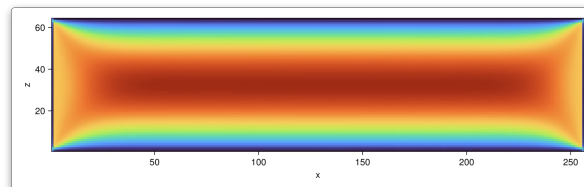


Figure 1: Poiseuille flow.

The lattice

The solver discretises the two-dimensional domain into a uniform grid of cells called a **lattice**. Lattice models use **DnQm** notation, where n is the number of spatial dimensions and m is the number of discrete velocities.

This solver uses the **D2Q9** model, so each cell holds 9 discrete velocity directions:

- One direction for particles at rest.
- Four directions for the orthogonal axes: up, down, left, and right.
- Four directions for the diagonals.

¹<https://academic.oup.com/gji/article/220/1/682/5574400>

Each cell therefore stores 9 particle distribution functions, $f_1 \dots f_9$, one per direction. Figure 2 shows the nine directions:

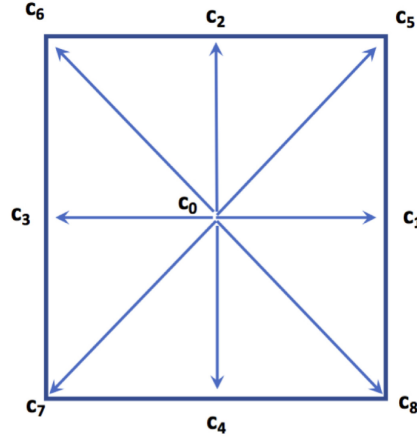


Figure 2: The D2Q9 lattice.

The sequential algorithm

The project provides the sequential solver in `src/run_seq.jl`. The solver evolves the fluid in time by repeating a single timestep for n_t steps. Each timestep runs the following four steps in order:

1. The **boundary step** drives the channel flow along x . It holds the leftmost column at density `rho_in` and the rightmost column at the lower density `rho_out`, by calling the helper `hold_density!(s, x, rho)` once for each of the two columns. The step is written inline in `run_seq`; `hold_density!(s, x, rho)` lives in `GeoFluidLBM.jl` and is available to your parallel solver.
2. **stream!** moves each particle distribution f_i to the neighbouring cell in direction i . If the upstream cell is a solid wall, the particles bounce back. Streaming is the only step that reads data from neighbouring cells, and it reaches exactly one cell in each direction.
3. **macroscopic!** recomputes the macroscopic fields from the current particle distributions `f`. The fields are the density `rho`, the momentum density `Pi`, and the velocity `u = Pi / rho`.
4. **equilibrium_collide!** performs the Bhatnagar–Gross–Krook (BGK) collision step. The function builds the local equilibrium `f_eq` from `rho` and `u`, then relaxes `f` towards it using $f \leftarrow f + (f_{eq} - f)/\tau_f$. The collision step uses only local data.

Data layout

The solver stores particle distributions as `f[direction, z, x]` with shape $(9, n_z, n_x)$:

- z is the vertical axis, which indexes rows.
- x is the horizontal axis, which indexes columns.

Julia stores arrays in column-major order, so the first index varies fastest and the last index varies slowest. Because x is the last index, the slice `f[:, :, x]` occupies one contiguous block of memory. That block holds an entire grid column.

3 Parallelisation approach

You must use a one-dimensional domain decomposition along x . The decomposition splits the grid into vertical strips of columns and assigns one strip to each MPI process. MPI identifies each process by its **rank**, an integer from 0 to $P - 1$, where P is the total number of processes. Each rank keeps the full grid height n_z .

You may assume that n_x is a multiple of the number of MPI processes P . Your implementation does not need to handle a leftover strip of columns. Every grid used by the test and by the benchmark sweep has a power-of-two n_x , and every rank count is a power of two, so this assumption always holds.

Your implementation must perform the following:

- **Exchange column data:** Exchange data between “neighbouring” ranks on every timestep to support cell streaming.
- **Apply boundary driving:** Apply the inlet and outlet driving only at the absolute ends of the global grid, not at the edges of individual strips.
- **Reconstruct the field:** Gather the final, full field on Rank 0 at the end of the run.

Further implementation details are found in Section 5 of the reference paper by Mora, Morra, and Yuen (2020). That section explains the decomposition, neighbour communication, and gather operations. **You may skip the paper completely and follow your own approach as long as it is based on a one-dimensional domain decomposition along the x direction.**

Note. In the paper, the calculations are performed only on the secondary ranks, while the main rank sends and receives data, and creates output files. **We highly recommend to not follow this approach and assign a subdomain to all ranks including the main rank.** In this way, you get potentially better load balance and there is no need to handle a leftover strip of columns.

Note. When implementing the neighbor exchange, it is highly recommended to pre-allocate the MPI request objects with function `MPI.Request` at the start of the simulation, before starting the time steps. Then, you can reuse these request objects at each time step, when calling `MPI.Isend` and `MPI.Irecv!`. This is potentially faster than creating new request objects at each time step. See the documentation of these functions for further details.

4 Tasks

Complete the following four tasks.

Implement

Implement `run_par(cfg, comm)` in `src/run_par.jl` by following the approach detailed in Section 3. Your solver must meet the following two requirements:

- **Match the sequential contract:** Rank 0 should return `(timings::Dict, state::State)`, matching the output of `run_seq`. Ensure that Rank 0 holds the full gathered field within the `state` variable.
- **Record communication time:** Include a "comm" entry in the `timings` dictionary to store the time the solver spends communicating. Calculate the compute time using the formula: `compute = total - comm`.

Answer the required questions and describe your implementation in Section 1 of the report template.

Verify

The project provides a test that validates the following two conditions:

- Both solvers reproduce the analytical Poiseuille parabola.
- The `run_par` solver matches the `run_seq` solver to round-off precision.

Note. The test must print **PASS** when running on both 2 ranks and 4 ranks. Passing on only 2 ranks is insufficient because it only exercises edge ranks, missing the interior case.

Run the test on DAS-5. See the `README.md` for the commands. Report the verification outcome in Section 2 of the report template.

Benchmark

Run the strong-scaling benchmark on the DAS-5 cluster using the provided job script `benchmark/benchmark_job.sh`. The script sweeps each grid over $P \in \{1, 2, 4, 8, 16\}$ ranks for the three grid sizes listed in table 1. Each run writes one JSON record to `benchmark/results/`.

Grid	n_x	n_z	n_t
Small	256	256	50
Medium	512	512	50
Large	1024	1024	50

Table 1: Fixed problem sizes for the strong-scaling benchmark.

Note. Do not change the problem sizes. Each grid must remain constant as P grows.

Describe your experimental setup and present your measurements in Section 3 of the report template.

Analysis

Extract all timing values from the `timings` dictionary within the JSON records. (Note: The benchmark script automatically retains the fastest time from three repeated runs). Use the $P = 1$ record as your sequential baseline (T_1) for each grid.

The table 2 lists the metrics you must compute for every grid and rank count:

Metric	Definition	Source
Runtime T_P	measured directly	<code>timings["total"]</code>
Speedup $S(P)$	T_1/T_P	computed
Efficiency $E(P)$	$S(P)/P$	computed
Communication fraction	<code>comm/total</code>	computed

Table 2: Required metrics, per grid and rank count.

Present the completed tables and figures in Section 3 of the report template, which specifies their required format. Answer the analysis questions in Sections 4 and 5.

5 Submission

Submit one `.zip` file per group to Canvas. Name the file `PLSPS_group_[NUMBER].zip`.

Deadline: Thursday October 22nd, 21:00.

Note. Automatic scripts check every submission, so follow these instructions exactly.

Required files

Create a folder called `PLSPS_group_[NUMBER]` and place exactly the following two files inside the folder.

- **Parallel implementation:** `run_par.jl`
- **Report:** `Report_group_[NUMBER].pdf`

Then, compress the folder into a `.zip` file. Upload this `.zip` file to Canvas.

Rules

The following rules apply to your submission:

- **Report:** Follow the provided template. Limit the document to 5 pages, including tables and figures.
- **Implementation:** Build the solver with the Julia MPI package. Ensure it runs on DAS-5 with the provided job scripts without modification.
- **Groups:** As stated in the Overview, you work in pairs. Form your group on the Canvas groups page before submitting, as described on the course page.
- **Resubmission:** You may resubmit before the deadline. We mark only the last submission.
- **Late submissions:** We do not accept late submissions.
- **AI tools:** **Using AI is not needed at all to complete the assignment.** The less you use AI, the more you will practice and learn. You are allowed to use AI tools if you wish to assist you with the implementation, the formatting of figures and tables, and the proofreading of the report (fixing typos and enhancing phrasing and formatting). **It is strictly forbidden to run AI tools on DAS-5.** The results you present in the report must be based on the data you obtained on DAS-5 by running your own code. You remain fully accountable for everything you submit and must be able to explain all aspects of your work to the teachers. **Disclose any AI tool usage and its purpose in your report.**

Submissions that do not follow these instructions may receive a penalty.

Anti-plagiarism rules

All forms of plagiarism are strictly prohibited. These include, but are not limited to:

- using code from the Internet
- making code publicly available on the Internet (e.g., on github)
- using code from fellow students,
- sharing code with fellow students
- using code from former students
- using AI beyond what is allowed in “AI tools” item above.

6 Contact

Ask questions on the Canvas discussion page. This is the main contact channel: post there so the whole class benefits from the answer, and check it before posting in case your question is already answered.

For private matters only, email one of the teaching assistants:

- Olin Feist
(o.v.s.feist@student.vu.nl)
- Gabriel Gil Pichelbauer
(g.j.gilyanez@student.vu.nl)
- Mahesh Karigoudar
(m.karigoudar@student.vu.nl)
- Maria Couceiro Feio De Almeida Penalva
(m.couceirofeiodealmeidapenalva@student.vu.nl)